Lecture 4 Compendium: Robotics in Agriculture

Module 2: Technologies of IoT in Crop Production

Dr. Kadeghe Fue
PhD Agroecology

Compendium for Advanced Study

Contents

1 Introduction: From Mechanization to Autonomy **2**

2 Robot Kinematics: The Mathematics of Motion **2**
2.1 Differential Drive Kinematics 2
2.2 Ackermann Steering (Tractors) 3
2.3 Python Simulation: Differential Drive 3

3 Control Theory: The PID Controller **4**
3.1 The Control Law 4
3.2 Python Implementation: PID Steering 4

4 Perception: Sensors & SLAM **5**
4.1 Key Sensors 5
4.2 SLAM (Simultaneous Localization and Mapping) 5

5 Path Planning: Coverage Algorithms **5**
5.1 Boustrophedon Pattern 6
5.2 A* (A-Star) Algorithm 6

6 Middleware: Robot Operating System (ROS) **6**
6.1 Core ROS Concepts 6
6.2 Example ROS Graph 6

7 Review Questions & Research Assignment **6**

1 Introduction: From Mechanization to Autonomy

The history of agricultural engineering is the history of energy substitution.

- **Mechanization (Ag 2.0):** Substituting animal muscle with internal combustion engines (ICE). The operator is still the "brain" and "sensor" of the machine.
- **Automation (Ag 3.0):** Systems that perform repetitive tasks automatically (e.g., Auto-steer tractors). The operator supervises.
- **Autonomy (Ag 5.0):** Robots that perceive their unstructured environment, make decisions, and execute tasks without human intervention.

The Research Challenge

Industrial robots (like those building cars) work in structured, deterministic environments. Agricultural robots must work in **unstructured, stochastic environments** involving mud, rain, variable lighting, and delicate biological targets. This is the "Unstructured Environment Problem."

2 Robot Kinematics: The Mathematics of Motion

To program a robot to move from Point A to Point B, we must model its physical constraints.

2.1 Differential Drive Kinematics

Most small ag-robots (e.g., weeding bots) use two independent drive wheels (Differential Drive).

State Vector: $q = [x, y, \theta]^T$

- x, y: Global position
- θ: Heading (Orientation)

Control Inputs:

- v: Linear velocity (speed)
- ω: Angular velocity (rotation rate)

The **Forward Kinematics** equations describe how the robot's state changes over time:

$$\dot{x} = v\cos(\theta) \tag{1}$$
$$\dot{y} = v\sin(\theta) \tag{2}$$
$$\dot{\theta} = \omega \tag{3}$$

If we control the left wheel speed (v_L) and right wheel speed (v_R), with wheel radius r and wheelbase (distance between wheels) L, then:

$$v = \frac{r}{2}(v_R + v_L) \tag{4}$$
$$\omega = \frac{r}{L}(v_R - v_L) \tag{5}$$

2.2 Ackermann Steering (Tractors)

Conventional tractors use Ackermann steering (car-like). They have a **Non-Holonomic Constraint**: they cannot move sideways.

$$\dot{x} = v\cos(\theta) \tag{6}$$

$$\dot{y} = v\sin(\theta) \tag{7}$$

$$\dot{\theta} = \frac{v}{L}\tan(\phi) \tag{8}$$

Where ϕ is the steering angle of the front wheels.

2.3 Python Simulation: Differential Drive

A class simulating a robot's movement given wheel velocities.

```python
import numpy as np
import matplotlib.pyplot as plt

class AgRobot:
    def __init__(self, x=0, y=0, theta=0, L=0.5, r=0.1):
        self.x = x          # X position (m)
        self.y = y          # Y position (m)
        self.theta = theta  # Heading (rad)
        self.L = L          # Wheelbase (m)
        self.r = r          # Wheel radius (m)
        self.history = {'x': [], 'y': []}

    def update(self, v_left, v_right, dt):
        """
        Forward Kinematics Update
        v_left, v_right: Angular velocity of wheels (rad/s)
        dt: Time step (s)
        """
        # Linear velocity of wheels
        vl = v_left * self.r
        vr = v_right * self.r

        # Robot frame velocities
        v = (vr + vl) / 2.0
        omega = (vr - vl) / self.L

        # Global frame updates (Integration)
        self.x += v * np.cos(self.theta) * dt
        self.y += v * np.sin(self.theta) * dt
        self.theta += omega * dt

        # Store history
        self.history['x'].append(self.x)
        self.history['y'].append(self.y)

# Simulation
robot = AgRobot()
dt = 0.1
# Drive in a circle: Right wheel faster than left
for t in np.arange(0, 10, dt):
    robot.update(v_left=5.0, v_right=6.0, dt=dt)

plt.plot(robot.history['x'], robot.history['y'])
plt.title("Robot Trajectory: Circular Path")
plt.xlabel("X (m)")
plt.ylabel("Y (m)")
```

```
plt.axis('equal')
plt.grid(True)
plt.show()
```

Listing 1: Differential Drive Kinematics Simulation

3 Control Theory: The PID Controller

Once we have the kinematics, we need a control algorithm to keep the robot on the path. The most fundamental controller is the **PID (Proportional-Integral-Derivative)**.

3.1 The Control Law

The goal is to minimize the **Cross-Track Error (CTE)**, denoted as $e(t)$, which is the distance between the robot and the desired path.

The control output $u(t)$ (usually the steering angle) is calculated as:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt} \tag{9}$$

- **Proportional (K_p):** Steer harder if the error is large. "Correct the present."
- **Integral (K_i):** Steer harder if the error persists over time (e.g., due to wheel alignment bias). "Correct the past."
- **Derivative (K_d):** Steer less if we are approaching the line quickly to avoid overshooting. "Predict the future."

3.2 Python Implementation: PID Steering

```
class PID:
    def __init__(self, Kp, Ki, Kd):
        self.Kp = Kp
        self.Ki = Ki
        self.Kd = Kd
        self.prev_error = 0
        self.integral = 0

    def compute(self, cte, dt):
        """
        cte: Cross Track Error (distance from path)
        dt: Time step
        """
        # Proportional term
        P = self.Kp * cte

        # Integral term
        self.integral += cte * dt
        I = self.Ki * self.integral

        # Derivative term
        derivative = (cte - self.prev_error) / dt
        D = self.Kd * derivative

        self.prev_error = cte

        # Total Control Output (Steering Angle)
        steering_angle = P + I + D
```

```
        return steering_angle

# Tuning Example
# High Kp -> Oscillates (Zig-Zag)
# High Kd -> Dampens oscillation
pid = PID(Kp=0.2, Ki=0.001, Kd=3.0)
```

Listing 2: PID Controller Implementation

4 Perception: Sensors & SLAM

"Perception" is the ability of the robot to understand its environment.

4.1 Key Sensors

1. **LiDAR (Light Detection and Ranging):** Uses laser pulses to create a 3D point cloud. High precision, works in dark, expensive. Excellent for obstacle avoidance.
2. **Stereo Cameras:** Uses two cameras to estimate depth (disparity map). Cheaper, rich data (color), computationally heavy.
3. **IMU (Inertial Measurement Unit):** Accelerometers and Gyroscopes. Measures acceleration and rotation. Drifts over time.
4. **RTK-GPS:** Real-Time Kinematic GPS. Provides cm-level accuracy but fails under tree canopies.

4.2 SLAM (Simultaneous Localization and Mapping)

In an unknown field, the robot must figure out: 1. Where am I? (Localization) 2. What does the world look like? (Mapping)

This is a "Chicken and Egg" problem. We solve it using Probabilistic Robotics, typically the **Extended Kalman Filter (EKF)** or **Particle Filters**.

The Bayes Filter Equation:

$$p(x_t|z_{1:t}, u_{1:t}) = \eta p(z_t|x_t) \int p(x_t|u_t, x_{t-1}) p(x_{t-1}) dx_{t-1}$$

Where:

- $p(x_t|\dots)$: Belief of current position.
- $p(z_t|x_t)$: Measurement Model (Likelihood of seeing sensor data z given position x).
- $p(x_t|u_t, x_{t-1})$: Motion Model (Likelihood of being at x_t given previous pos and movement u).

5 Path Planning: Coverage Algorithms

In agriculture, we rarely just go "A to B". We need to cover the whole field (spraying, harvesting). This is the **Coverage Path Planning (CPP)** problem.

5.1 Boustrophedon Pattern

The standard "Ox-turning" pattern. The robot traverses the field in parallel lines.

Challenges:

- **Headlands:** The turning area at the end of the row. Robots perform Ω-turns or Fish-tail turns to align with the next row.
- **Optimality:** Minimizing non-working travel distance (deadheading).

5.2 A* (A-Star) Algorithm

For obstacle avoidance (e.g., driving around a rock), we use A*. It finds the shortest path on a grid graph.

$$f(n) = g(n) + h(n)$$

- $g(n)$: Cost from start to node n.
- $h(n)$: Heuristic estimated cost from n to goal (usually Euclidean distance).

6 Middleware: Robot Operating System (ROS)

Research robotics runs on **ROS**. It is not an OS (like Windows), but a middleware framework that handles communication between hardware and code.

6.1 Core ROS Concepts

- **Nodes:** Independent processes performing a task (e.g., `camera_driver`, `path_planner`, `motor_controller`).
- **Topics:** Named buses over which nodes exchange messages.
  - **Publish/Subscribe Pattern:** `camera_driver` *publishes* images to `/camera/raw`. `path_planner` *subscribes* to `/camera/raw`.
- **Messages:** Strict data structures (e.g., `geometry_msgs/Twist` for velocity).

6.2 Example ROS Graph

```
[Camera Node] --(Image)--> [Computer Vision Node] --(Weed Loc)--> [Planner Node]
                                                                        |
                                                                  (Velocity Cmd)
                                                                        v
                                                                  [Motor Controller]
```

7 Review Questions & Research Assignment

1. **Kinematics:** A differential drive robot has a wheel radius of 10cm and a wheelbase of 50cm. If the left wheel spins at 10 rad/s and the right wheel at 20 rad/s, what is the robot's linear velocity and angular velocity?
2. **Control:** Explain why a purely Proportional (P) controller might result in the robot oscillating around the target path forever. How does the Derivative (D) term fix this?

3. **Perception:** Why is LiDAR preferred over Cameras for night-time operations, but Cameras preferred for disease detection?
4. **Design Challenge:** Design a "Swarm" system for weeding 100 hectares.
    - Constraint: Robots must be small (solar charged).
    - Question: How do you handle the "charging logistics" and "central coordination"?

References

1. Corke, P. (2017). *Robotics, Vision and Control*. Springer.
2. Thrun, S., Burgard, W., & Fox, D. (2005). *Probabilistic Robotics*. MIT Press.
3. Duckett, T., et al. (2018). *Agricultural Robotics: The Future of Robotic Agriculture*. UK-RAS White Paper.
4. ROS.org Documentation. `http://wiki.ros.org/`